TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1

NGÀNH: CÔNG NGHỆ THÔNG TIN

Bài Tập 3:Kiểm Thử Và Đóng Gói Prototype

MÃ LUỒNG NGHIỆP VỤ L10 – TỰ ĐỘNG PHÂN LOẠI YÊU CẦU BẢO HÀNH – TRACK AI

SVTH: Lê Hoàng Hiệp **MSSV:** 2374802010139

LỚP: 261_71ITGR40203_05

NGÀY NỘP : 13/11/2026

LINK GITHUB REPO: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification

Tag phiên bản: v1.0

TP. Hồ Chí Minh – Năm 2026

Mục Lục

MỤC 1: TEST PLAN RÚT GỌN 3
1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm thử 3
2. Loại test và số lượng mỗi tầng 3
3. Tiêu chí thành công ĐO ĐƯỢC 3
4. Môi trường và dữ liệu thử 3
MỤC 2: TEST REPORT (10 TEST CASE) 4
MỤC 3: BUG LOG 7
MỤC 4: DEPLOYMENT GUIDE 9
1. Requirements 9
2. Setup steps 9
3. Environment variables 9
4. Sample data 9
5. Verify it works 9
MỤC 5: CHECKLIST BẢO MẬT 10
MỤC 6: MODEL CARD 11
1. Basic Information 11
2. Intended Use 11
3. Out-of-Scope Uses 11
4. Training Data 11
5. Architecture and Processing 11
6. Evaluation Metrics 11
7. Baseline 11
8. Measured Bias 11
9. Risks and Limitations 11
10. Maintenance 12
MỤC 7: ẢNH CHỤP MINH CHỨNG 13
PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI 14

MỤC 1: TEST PLAN RÚT GỌN

1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm thử

- Kiểm thử: Hàm làm sạch văn bản (lam_sach), quá trình chia dữ liệu (chống rò rỉ), quá trình huấn luyện (tính tái lập), API dự đoán (dữ liệu đúng/sai).
- Không kiểm thử: Giao diện web (vì dự án chỉ làm API), hiệu năng phần cứng (thuộc phạm vi môn học khác).

2. Loại test và số lượng mỗi tầng

- Unit Test: 5 test case (tập trung vào hàm làm sạch và API).
- Integration/ML Test: 5 test case (kiểm tra rò rỉ dữ liệu, tính tái lập, giới hạn đầu vào).
- Tổng cộng: 10 test case.
- Kỹ thuật chọn dữ liệu: Phân vùng tương đương và giá trị biên (mô tả rỗng, mô tả quá ngắn).

3. Tiêu chí thành công ĐO ĐƯỢC

- 100% test case PASS khi chạy lệnh python -m pytest -q.
- Mô hình đạt F1-macro > 0.7 trên tập Test.
- API không bị sập (trả về mã 500) khi gặp dữ liệu sai.

4. Môi trường và dữ liệu thử

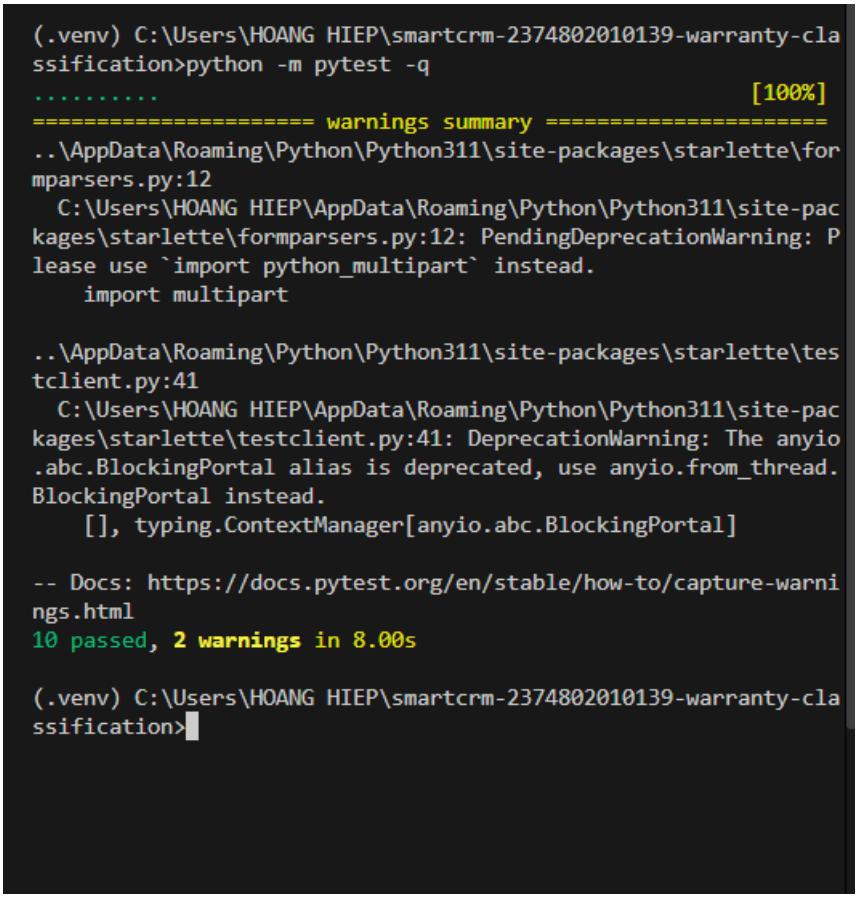
- Môi trường: Chạy trên Docker container (môi trường ảo độc lập).
- Dữ liệu thử: Dữ liệu được sinh tự động trong code test (không phụ thuộc file có sẵn trên máy).

MỤC 2: TEST REPORT (10 TEST CASE)

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
TC-01	Hàm lam_sach sẵn sàng	Gọi hàm làm sạch với văn bản thường	"Màn Hình Bị SỌC Xanh!@#"	Trả về "màn hình bị sọc xanh"	Cao	Đúng như mong đợi	PASS
TC-02	Hàm lam_sach sẵn sàng	Gọi hàm làm sạch với văn bản chứa SĐT/Email	"Gọi 0912345678, mail a@b.com"	Che thông tin thành <sdt>, <email>	Cao	Đúng như mong đợi	PASS
TC-03	Hàm lam_sach sẵn sàng	Gọi hàm làm sạch với chuỗi rỗng	""	Trả về "", không sập	Trung bình	Đúng như mong đợi	PASS
TC-04	Hàm lam_sach sẵn sàng	Gọi hàm làm sạch với giá trị None	None	Trả về "", không ném ngoại lệ	Trung bình	Đúng như mong đợi	PASS
TC-05	Dữ liệu văn bản thô	Kiểm tra giữ dấu và thay thế serial	"SN10293A màn hình vỡ"	Chứa <serial>, giữ	Cao	Đúng như	PASS

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
				nguyên dấu		mong đợi	
TC-06	Đã có tập Train/Test	Kiểm tra rò rỉ dữ liệu (Leakage)	train_texts, test_texts	Vectorizer không chứa từ chỉ có ở Test	Cao	Đúng như mong đợi	PASS
TC-07	Môi trường sẵn sàng	Chạy script train.py 2 lần liên tiếp	configs/train.yaml	2 file metrics.json giống hệt nhau	Cao	Đúng như mong đợi	PASS
TC-08	API đang chạy (Docker)	Gọi API với dữ liệu hợp lệ	{"mo_ta": "Máy sạc không vào điện"}	Mã 200, trả về top 3 nhãn và độ tin cậy	Cao	Đúng như mong đợi	PASS
TC-09	API đang chạy (Docker)	Gọi API với mô tả rỗng	{"mo_ta": ""}	Mã 422 (Lỗi validation)	Cao	Đúng như mong đợi	PASS
TC-10	API đang chạy (Docker)	Gọi API với mô tả quá ngắn	{"mo_ta": "abc"}	Mã 422 (Lỗi validation)	Cao	Đúng như	PASS

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
						mong đợi	



MỤC 3: BUG LOG

BUG-001 · Critical — Ứng dụng Docker bị crash ngay khi khởi động

1. Mã và mức độ: BUG-001 · Critical.
2. Phát hiện bởi: Kiểm thử thủ công quá trình chạy Docker (docker compose logs model).
3. Triệu chứng: Container tự động tắt. Log báo ValueError: Unknown level: 'info'.
4. Cách tái lập: Chạy docker compose up -d, sau đó xem log.
5. Nguyên nhân: Biến môi trường LOG_LEVEL là chữ thường (info), thư viện logging yêu cầu chữ IN HOA.
6. Cách sửa: Sửa hàm cau_hinh_log() trong src/log_config.py, thêm .upper().
7. Commit và xác nhận: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification/commit/9d3a2b3 - Chạy lại Docker báo healthy.

BUG-002 · Major — Lỗi làm sạch văn bản ăn nhầm chữ thành Email

1. Mã và mức độ: BUG-002 · Major.
2. Phát hiện bởi: Test case test_lam_sach_luong_dung.
3. Triệu chứng: Hàm trả về "màn hình bị sọc email" thay vì "màn hình bị sọc xanh".
4. Cách tái lập: Chạy pytest -q, test case 1 FAILED.
5. Nguyên nhân: Regex email \S+@\S+ quá lỏng lẻo, khớp nhầm với cụm "Xanh!@#".
6. Cách sửa: Thắt chặt Regex email thành r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'. Giữ nguyên dấu tiếng Việt.
7. Commit và xác nhận: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification/commit/f76300d - Chạy lại pytest PASS 100%.

BUG-003 · Major — Lỗi build Docker do thư viện đặc thù của Windows

1. Mã và mức độ: BUG-003 · Major.
2. Phát hiện bởi: Chạy docker compose up -d --build tại thư mục trống.
3. Triệu chứng: Build thất bại, báo lỗi linker 'cc' not found khi biên dịch pywinpty.
4. Cách tái lập: Xóa cache Docker, build lại.
5. Nguyên nhân: requirements.txt chứa pywinpty (chỉ dành cho Windows). Docker Linux không có trình biên dịch C/Rust.
6. Cách sửa: Xóa dòng pywinpty khỏi requirements.txt.
7. Commit và xác nhận: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification/commit/95c1019 - Build thành công, container healthy.

BUG-004 · Major — Lỗi build Docker thất bại do thiếu thư mục chứa file báo cáo

1. Mã và mức độ: BUG-004 · Major (Quá trình đóng gói sản phẩm thất bại, không thể triển khai hệ thống).
2. Phát hiện bởi: Phép thử máy trắng (White-box test) - Clone repo vào thư mục trống `C:\test-bt3` và chạy lệnh `docker compose up -d --build`.
3. Triệu chứng: Quá trình build Docker bị dừng đột ngột ở bước `RUN python -m src.models.train`. Log hiển thị lỗi `FileNotFoundError: [Errno 2] No such file or directory: 'reports/metrics.json'`.
4. Cách tái lập: Xóa thư mục dự án hiện tại, clone lại repo từ GitHub về một thư mục hoàn toàn mới, sau đó chạy lệnh `docker compose up -d --build`.
5. Nguyên nhân: Thư mục `reports/` đã bị đưa vào file `.gitignore` (do chứa các file kết quả sinh ra tự động) nên không tồn tại khi clone về máy mới. Khi script `train.py` chạy xong, hàm lưu kết quả cố gắng ghi file `metrics.json` vào thư mục `reports/` nhưng thư mục mẹ này chưa được khởi tạo, dẫn đến lỗi sập tiến trình.
6. Cách sửa: Bổ sung thư viện `os` và thêm dòng lệnh `os.makedirs('reports', exist_ok=True)` vào file `src/models/train.py` (ngay trước đoạn code thực hiện ghi file `metrics.json`) để tự động tạo thư mục nếu nó chưa tồn tại.
7. Link commit và xác nhận: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification/commit/21fd2ff - Chạy lại `docker compose up -d --build` từ thư mục trống -> Build thành công, container báo `healthy`.

MỤC 4: DEPLOYMENT GUIDE

1. Requirements

- Docker Desktop (Engine 24+, Docker Compose v2)
- Git
- Port 8080 must be free.

2. Setup steps

1. git clone --branch v1.0 https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification.git
2. cd smartcrm-2374802010139-warranty-classification
3. docker compose up -d --build

3. Environment variables

Variable	Meaning	Sample Value	Required
LOG_LEVEL	Logging level for the API	INFO	No (default: INFO)
NGUONG_TIN_CAY	Confidence threshold for prediction	0.45	No (default: 0.45)

4. Sample data

The model is trained automatically during the Docker build using data/issue_descriptions.csv. No manual data loading is required.

5. Verify it works

1. Check container status: docker compose ps (STATUS must be healthy).
2. Test API health: curl http://localhost:8080/health -> expected {"status":"ok","phien_ban_mo_hinh":"1.0.0"}
3. Test prediction:
   curl -X POST http://localhost:8080/du-doan -H "Content-Type: application/json" -d "{\"mo_ta\": \"Màn hình bị sọc\"}"
   -> expected JSON with predicted label MAN_HINH.
4. Run tests: docker compose exec model python -m pytest -q -> expected 10 passed.

MỤC 5: CHECKLIST BẢO MẬT

STT	Hạng mục	Trạng thái	Minh chứng / Lý do
1	Không có secret trong repo	ĐÃ LÀM	Chạy git log --all --oneline -- .env không ra kết quả.
2	Dữ liệu vào được kiểm tra	ĐÃ LÀM	Pydantic schema giới hạn mo_ta từ 5-2000 ký tự. Test case trả về 422 khi sai.
3	Kiểm soát truy cập	KHÔNG ÁP DỤNG	Module AI không có chức năng đăng nhập người dùng.
4	Mật khẩu băm/mã hóa	KHÔNG ÁP DỤNG	Không có chức năng lưu mật khẩu.
5	Thông báo lỗi không lộ nội bộ	ĐÃ LÀM	API trả về mã 422/500 với message chung, không trả stack trace.
6	Nhật ký an toàn	ĐÃ LÀM	Log chỉ ghi do_dai_mo_ta, nhan_du_doan, do_tin_cay, KHÔNG ghi nội dung mô tả.
7	Phụ thuộc ghim phiên bản	ĐÃ LÀM	File requirements.txt đã dùng ==.

MỤC 6: MODEL CARD

1. Basic Information

- Model name: warranty-classification v1.0
- Author: Le Hoang Hiep (MSSV: 2374802010139)

2. Intended Use

Suggests an issue category to intake staff based on free-text descriptions. Staff makes the final decision.

3. Out-of-Scope Uses

- NOT for automatically assigning technicians.
- NOT for deciding warranty eligibility without human confirmation.
- NOT for languages other than Vietnamese.

4. Training Data

- Source: issue_descriptions.csv (2400 samples).
- Split: 70/15/15 (stratified), random_state=42.
- Label Distribution: MAN_HINH (32.8%), PIN (24.5%), etc.

5. Architecture and Processing

- Vectorizer: TF-IDF (ngram 1-2, max_features=20000).
- Model: ComplementNB (chosen via validation set comparison).
- Preprocessing: Lowercase, keep Vietnamese diacritics, mask Phone/Email.

6. Evaluation Metrics

- Test F1-macro: 0.8346 (Data v1.0, seed 42).
- Per-label F1: KHAC (0.81), MAN_HINH (0.90), NUOC_VAO (0.83), PHAN_MEM (0.77), PIN (0.86), SAC (0.85).

7. Baseline

- Dummy Classifier (Majority class): F1-macro = 0.059.

8. Measured Bias

- Measured by label. The minority label KHAC (5.5% of data) achieved an F1 of 0.81, showing no significant bias against the minority class. The confusion matrix shows confusion between PIN and SAC due to similar wording.

9. Risks and Limitations

- Similar wording: Descriptions for PIN and SAC are often confused. Mitigation: Staff reviews the top 3 predictions.

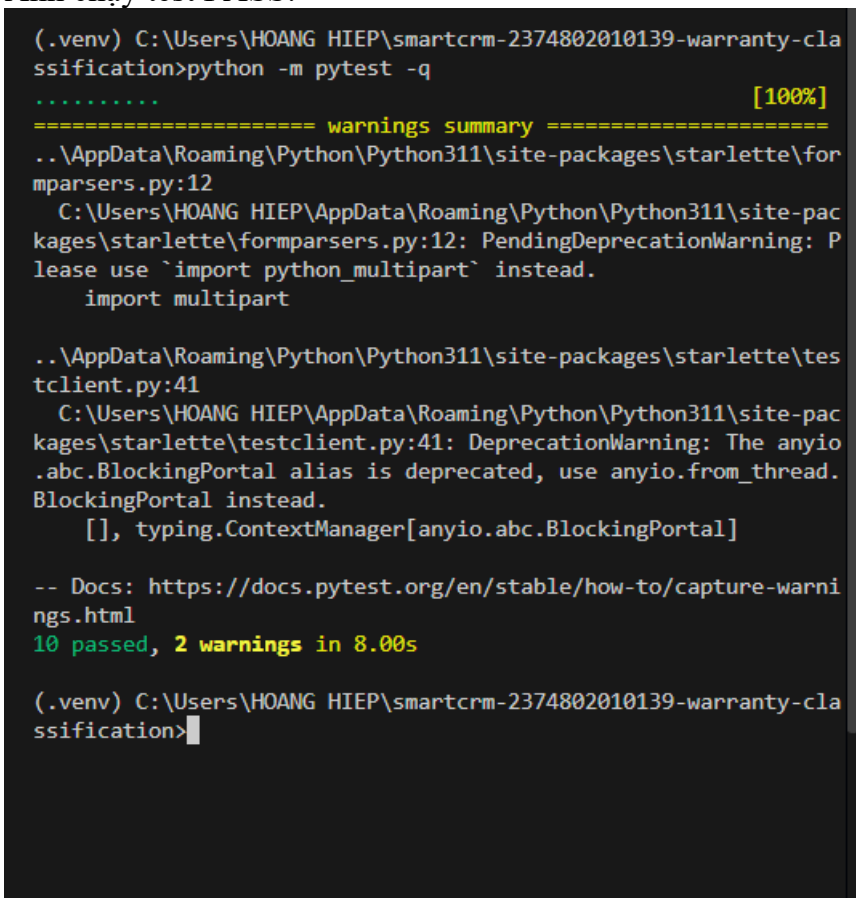
- Low confidence: If confidence is < 0.45, the system flags can_nguoi_xem_lai = true. Mitigation: Staff must choose manually.

10. Maintenance

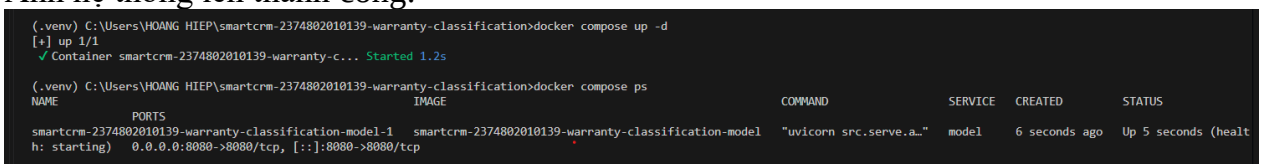
- Retrain when the staff correction rate exceeds 35%, or when a new issue category is added.

MỤC 7: ẢNH CHỤP MINH CHỨNG

1. Ảnh chạy test PASS:



2. Ảnh hệ thống lên thành công:



3. Ảnh phép thử máy trắng:



PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI

Công cụ	Dùng vào việc gì	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào
DeepSeek	Gợi ý Regex làm sạch văn bản	src/features/text.py, hàm lam_sach	Đã tự sửa lại Regex để giữ dấu tiếng Việt; viết 6 unit test kiểm tra lại. Chạy pytest pass 100%.
DeepSeek	Gợi ý cấu trúc FastAPI	src/serve/app.py	Đã đọc tài liệu chính thức của FastAPI; tự viết lại các endpoint /health và /du-doan.

Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp.
Lê Hoàng Hiệp – 2374802010139 – 13/11/2026